Module 2.0

Software Requirements Analysis and Modeling

WHAT IS REQUIREMENT?

A requirement is defined as a condition or capability to which a system must conform.

Requirement management is a systematic approach of

- finding,
- documenting,
- organizing and
- tracking the changing requirements of a system.

Requirement engineering helps in understanding the requirements of a problem.

TYPES OF REQUIREMENTS

In computer-based systems, types of requirements are-

1. **User requirements**
    - User requirements focus on the **needs and goals of the end users**.
2. **System requirements**

They set out the **system's functions, services and operational constraints** in detail.

TYPES OF SOFTWARE SYSTEM REQUIREMENTS

Software **system requirements (system's functions, services and operational constraints)** are classified as

- **functional** and
- **non-functional requirements**

Functional requirements

- define the **basic functions that the system must provide and focus** on the **needs and goals** of the end users and
- specify the **input and output behavior of a system**.

Examples of Functional requirements

- **Authentication of user** whenever he/she logs into the system.
- System must **shutdown in case of a cyber-attack**.

- A verification email or OTP is sent to user **whenever user registers for the first time** on software system.
- The **background color for all windows** in the application will be blue and have a hexadecimal RGB color value of 0x0000FF.

Non-functional requirements

Non-functional requirements define the **performance attributes** of a software system.

They help to **ensure good user experience and ease of operating the software**.

Types of Non-functional requirements

- Scalability
- Reliability
- Maintainability
- Security
- Flexibility
- Availability
- Reusability
- Portability

Examples of Non-functional requirements

- Users must change the initially assigned login password immediately after the first successful login.
- Employees never allowed to update their salary information.
- Every unsuccessful attempt by a user to access an item of data shall be recorded on an audit trail.
- A website should be capable enough to handle 10000 users without affecting its performance.
- The processing of each user request should be done within 10 seconds.
- The site should load in 3 seconds.
- The software should be portable. So moving from one OS to other OS does not create any problem.

OBJECTIVES OF REQUIREMENTS ANALYSIS

- To **establish and maintain agreement with the customers and other stakeholders** on **what the system should do**.
- To provide system developers with a **better understanding of the system requirements**.
- To **define the boundaries** (**scope**) of the system.
- To provide a basis for **estimating cost and time** to develop the system.
- To define a **user-interface for the system**, focusing on the needs and goals of the users.

The purpose of requirements analysis is to **specify the complete functional and non-functional software requirements** for the system.

REQUIREMENTS ENGINEERING

Requirements engineering is the **broad spectrum of tasks and techniques** that lead to an **understanding of requirements.**

It must be adapted to the **needs of 4 Ps** of software engineering doing the work**.**

These 4 Ps are

- project,
- product,
- process, and
- people.

Requirements engineering **provides the appropriate mechanism** for -

- understanding **what the customer wants**,
- **analysing** needs,
- assessing **feasibility**,
- negotiating a **reasonable solution**,
- specifying the **solution unambiguously**,
- **validating the specification**, and
- **managing the requirements** as they are transformed into an operational system.

Requirements engineering encompasses **seven distinct tasks:**

1) inception,
2) elicitation,
3) elaboration,
4) negotiation,
5) specification,
6) validation, and
7) management.

1. Inception is **understanding of**
   - the problem,
   - the people who want a solution,
   - the nature of the solution that is desired, and
   - preliminary communication and collaboration between the other stakeholders and the software team.
2. Elicitation is to identify **objectives for the system or product** are,
   - **what** is to be accomplished,
   - how the system or product fits into the **needs of the business**, and
   - finally, how the system or product is to be **used on a day-to-day basis**.
3. Elaboration is **refinement of information** obtained from the customer during inception and elicitation.
4. In negotiation,
   - Customers, users, and other stakeholders are asked to **rank requirements and then discuss conflicts in priority.**
   - Using an iterative approach that **prioritizes requirements**, assesses their **cost and risk**, and requirements are **eliminated, combined, and/or modified** so that each party achieves some measure of satisfaction.
5. Specification is a **written document** for specification is created with
   - a set of graphical models,
   - a formal mathematical model, and
   - a collection of usage scenarios, a prototype, or any combination of these.

6. **Validation –** It validates that
    - all software requirements for unambiguous (without any inconsistencies, omissions, and errors) and
    - the work products conform to the standards established for the process, the project, and the product.
7. Requirements management
    - helps the project team **identify, control, and track requirements** and **changes to requirements** at any time as the project proceeds.

Some of these tasks occur in parallel and all are adapted to the needs of the project.

REQUIREMENTS ELICITATION (REQUIREMENTS GATHERING)

Requirements elicitation (**requirements gathering**) consists of

- **problem solving,**
- **elaboration,**
- **negotiation,** and
- **specification.**

The following techniques can be applied **to collect and elicit relevant information** from the **stakeholders**:

1. interviews
2. document analysis/review
3. brainstorming
4. observation
5. prototyping
6. requirements workshop/Joint Application Development (JAD)
7. questionnaires/surveys

Cost and time are important factors when determining which method to be used. The method in gathering the requirements may vary depending **on the situation and constraints**, but using the various methods to **supplement each other** helps in achieving complete requirements.

1. Interviews-

Interviews are the **most common technique** for gathering requirements, as well as one of the **primary sources of requirements**.

There are different types of interviews and interview questions.

Types of interviews can be One-on-One interviews and Group interviews.

Interviews can be **structured interview or unstructured interviews.**

If the interviewer has a predefined set of questions, then it’s called **a structured interview.**

If the interviewer is not having any particular format or any specific questions, then it's called an **unstructured interview**.

2. Brainstorming

It is used to get as many ideas as possible from **group of people**.

Generally used to **identify possible solutions** to problems, and clarify details of opportunities.

3. Document Analysis / Reviews

Analyzing or reviewing existing documents can prove to be a useful technique in requirement gathering.

Reviewing the current process and documentation can help the analyst understand the business, or system, and its current situation.

Existing documentation will provide the analyst the titles and names of stakeholders who are involved with the system.

This will **help the analyst formulate questions** for interviews or questionnaires to ask of stakeholders, in order to gain additional requirements.

4. Observation

To get a better understanding of a user in their current work environment, the analyst may **observe the user themselves**.

User observation is helpful in assisting the analyst by getting a **full grasp of how the user interacts with the system, firsthand**.

5. Prototyping

Prototyping is used **to identify missing or unspecified requirements**.

In this technique, frequent **demos** are given to the client **by creating the prototypes** so that **client can get an idea of how the product will look like and work**.

Prototypes can be used to create **a mock-up of sites**, and describe the process using diagrams.

6. Requirements Workshop / Joint Application Design (JAD)

These are **structured meetings** involving end-users, project managers and other stake holders.

This is used to **define, clarify, and complete requirements**.

JAD is usually conducted in at a location other than the place where the people involved work. This helps to reduce distractions.

Some of the benefits of JAD include:

- Consolidation of work into a structured workshop
- Clarifies specification requirements in an environment of consensus

- Ties together the steps of the design process in a concise document
- Increases user satisfaction by directly involving users in the design process
- Builds commitment through the use of the executive sponsor

7. Questionnaires / Surveys

Questionnaires or surveys allow an analyst **to collect information from many people in relatively short amount of time.**

This is especially helpful when **stakeholders are spread out geographically**, or there are **many respondents** whose input will be needed to help establish system requirements.

When using questionnaires, the **questions should be focused and organized** by a feature or project objective.

Questionnaires should be not be too long, to ensure that users will complete them.

SOFTWARE REQUIREMENT SPECIFICATION (SRS) DOCUMENT

A Software Requirements Specification (SRS) is also known as **SRS report**, **software document**.

It is a **document** that

- describes the **nature of a project, software or application,**
- contains description of **all aspects of the software to be built that must be specified before the project is to commence** and
- is also known as **SRS report**, **software document**.

TEMPLATE for SRS DOCUMENT

These are a **set of guidelines to be followed** while preparing the software requirement specification document.

Different templates are available that can **serve as a guideline** for those who must create a complete SRS.

Contents of SRS template

- purpose
- scope
- functional requirements
- non-functional requirements
- software requirements
- hardware requirements
- environmental conditions required
- safety and security requirements
- software quality attributes of the project and etc.

IEEE Template for SRS

Table of Contents

Revision History

1. Introduction

1.1 Purpose
1.2 Document Conventions
1.3 Intended Audience and Reading Suggestions
1.4 Project Scope
1.5 References

2. Overall Description

2.1 Product Perspective
2.2 Product Features
2.3 User Classes and Characteristics
2.4 Operating Environment
2.5 Design and Implementation Constraints
2.6 User Documentation
2.7 Assumptions and Dependencies

3. System Features

3.1 System Feature 1
3.2 System Feature 2 (and so on)

4. External Interface Requirements

4.1 User Interfaces
4.2 Hardware Interfaces
4.3 Software Interfaces
4.4 Communications Interfaces

5. Other Non-functional Requirements

5.1 Performance Requirements
5.2 Safety Requirements
5.3 Security Requirements
5.4 Software Quality Attributes

6. Other Requirements

Appendix A: Glossary
Appendix B: Analysis Models
Appendix C: Issues List

Example SRS-

Problem statement for Library Information System (LIS)

- An institute has proposed to **develop a Library Information System (LIS)** for **students and employees.**
- LIS will enable the members **to borrow a book (or return it)** with ease while sitting at his desk/chamber.
- Issuing or returning books is **restricted to valid users** (members) of LIS only.
- The system also enables a member **to extend the date of his borrowing,** if no other booking for that particular book has been made.
- The librarian can **enter a new record into the system when a new book has been purchased**, or **remove a record in case any book is taken off the shelf**.
- Any non-member is free to use this system **to browse/search books online**.
- No password should not be stored in system **as a plain text**.
- The final software would a **web application** (using the recent HTML 5), used **within the institute LAN**.

Identification of functional requirements

The functional requirements of the system can be:

- **New user/member registration**
- **Search book**
- **User/member login**
- **Issue book**
- **Return book**
- **Re-issue book**
- **Add book**
- **Delete book**

Having identified the functional requirements, an identifier may be assigned to each of them for **future reference and verification**.
Following table shows the **identifier and priority for software requirements** as

Requirement No.	Requirement	Priority
R1	New user registration	High
R2	User Login	High
R3	Search book	High
R4	Issue book	High
R5	Return book	High
R6	Add book in record	High
R7	Remove book from record	High
R8	Re-issue book	Low

Identification of non-functional requirements

- **Performance Requirements:**
    - This system should remain **accessible 24x7**.
    - At least **50 users** should be able to access the system altogether **at any given time.**
- **Security Requirements:**
    - This system should be accessible **only within the institute LAN**.
    - The database of LIS should not store any password **in plain text - a hashed value has to be stored.**
- **Software Quality Attributes**
    - Reliability
    - Availability
    - Security
    - Portability
- **Database Requirements**
    - File Format (data shall be stored in **text-based flat files**)
    - Accessibility and security
- **Design Constraints:**
    - The LIS has to be developed as a web application, which should work with Firefox 5, Internet Explorer 8, Google Chrome 12.
    - The system should be developed using HTML 5.

REQUIREMENT MODEL

The requirement model is a **snapshot of requirements at any given time.**

The **model changes dynamically** as the system is built, and other stakeholders understand more about what they really require.

The requirements model achieves **three primary objectives**:

(1) to describe **what the customer wants**,

(2) to establish a basis for the **creation of a software design**, and

(3) to define a **set of requirements that can be validated** once the software is built.

The requirements modeling action results in one or more of the **following types of models**:

1. Scenario-based models
2. Class-oriented models
3. Flow-oriented models
4. Behavioral models

These models provide a software designer with **information that can be translated** to **architectural, interface,** and **component-level designs**.

Elements of the Requirements Model

1. Scenario-based elements

- The system is described from the user's point of view using a **scenario-based approach**.
- They are represented using **actors**, **use cases** and their corresponding **use-case diagrams.**

2. Class-based elements

- Each usage scenario implies a **set of objects** that are manipulated as an actor interacts with the system.
- **These objects are categorized into classes** (a collection of things that have similar attributes and common behaviors).
- They are represented using **class diagrams and collaboration diagrams**.

3. Behavioral elements

- They represent the **behavior of a system**.
- They are represented using **state diagram** and **sequence diagrams**.

4. Flow-oriented elements

- In a computer-based system, information gets **transformed into different forms as it flows** through system.

- The system **accepts input** in a variety of forms, applies functions to **transform it**, and **produces output** in a variety of forms, regardless of size and complexity.
- They are represented using **data flow diagrams.**

SCENARIO-BASED APPROACH

- The system is described from the user's point of view using a **scenario-based approach**.
- **Scenario-based elements** are represented using **actors, use cases** and their corresponding **use-case diagrams.**

DEVELOPING USE CASES

Use Cases

After **requirements are gathered**, the developers and end users can create **a set of scenarios that identify a usage of the system** to be constructed.

These scenarios are called **use cases.**

The **use cases** provide a **description of how the system will be used**.

A **use case** is a **description about how an end user** (playing one of a number of possible **roles**) **interacts with the system under a specific set of circumstances**.

A use case depicts the software or system **from the end user's point of view**.

Actors in use cases

The first step in writing a use case is to define the set of **"actors"** that will be involved in the system usage.

Actors are the **different people or non-human entities/devices** that use the system or product within the context of the function and behaviour that is to be described.

An actor is the **UML term to identify someone or something that communicates with the system or product** and that is **external to the system** itself.

Every actor has **one or more goals** when using the system.

Representing actors in use cases

Symbol for representing actor is "**stick man**" icon with the name of the actor below the icon.


Student


Employee


ATM


DB Server

Actors represent the **roles that people or devices/ non-human** play as the system operates.

Human as actors can be

- customer (bank, shop, hotel)
- member (library, sports_club, association)
- user (ATM, vending machine)
- patient, doctor, nurse
- student , teacher

Non-human entities/devices as actors can be

- bank, hospital, college, organisation
- server
- ATM machine
- payment system (cash, card, online)
- web client

Types of Actors

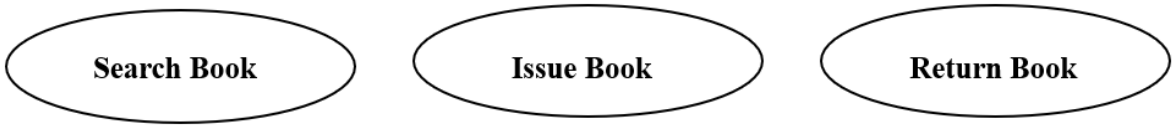
- **Primary actors-**
  - They interact to achieve required system function and derive the intended benefit from the system. They work **directly and frequently with the software**.
- **Secondary/ supporting actors –**
  - They **support the system** so that primary actors can do their work.

Identifying Use cases

- Use cases **capture the functional requirements** of a system.
- Use cases describe **what happens in the system when an actor interacts** with the system to execute the use case.
- They specify the behavior of a system or some subset of a system.
- Each of use cases represents a specific flow of events.

Use cases are graphically **represented as an oval** with the name of its functionality written in oval.

Use cases for library management system can be represented as



USE CASES DIAGRAMS

Diagrams with **actors, use cases, and relationships** among them are called **use-case diagrams** and illustrate relationships in the use-case model.

It describes a **system's requirements** in terms of use cases.

A use-case model

- is a model of the system's **intended functions and its surroundings**, and
- serves as a **contract between the customer and the developers**.
- A use case is **triggered by an actor**.
- Actors and use cases are connected through **binary associations indicating that the two communicates through message passing.**
- An actor must be associated with **at least one use case**.
- Similarly, a **given use case must be associated with at least one actor**.

Example- Course Registration System

PROBLEM STATEMENT:

This system will maintain a course registration database which will allow the student to view the course details and to register the course.

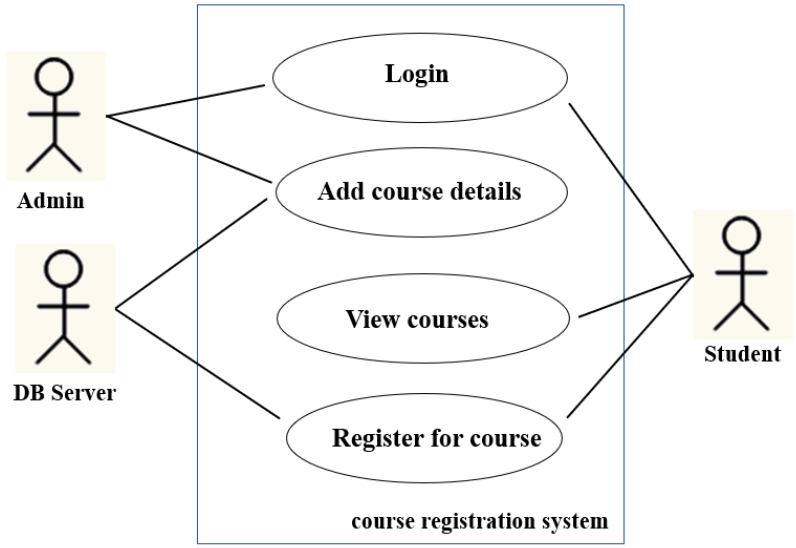
Identify actors involved:

- **Student**: User who view courses and register the course
- **Admin**: To add courses and details

Identify use-cases :

- Login
- Add courses
- View courses
- Register for course

Use case diagram for course registration system



TYPES of USE CASE RELATIONSHIPS

Three types of relationships exist among use cases:

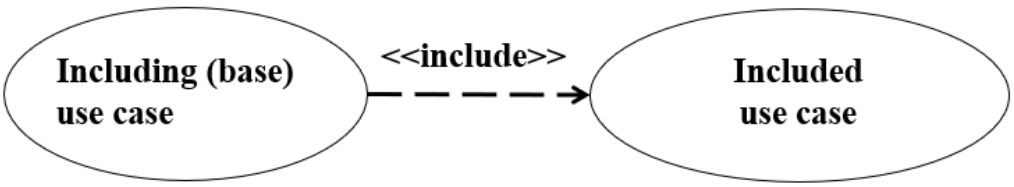
- Include relationship
- Extend relationship
- Generalization/specialization relationship

Include relationship

Include relationships are used **to depict common behaviour** that are shared by multiple use cases.

Notation for include relationships

Include relationship is depicted by a **dashed arrow** with a **«include»** stereotype from the **including use case to the included use case**.

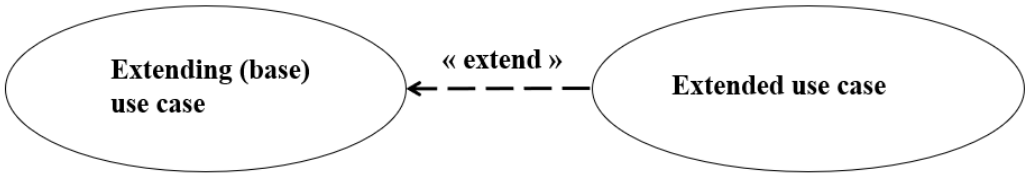


Extend relationship

The extend relationship is used to **include optional behavior** from an extending use case in an extended use case.

Notation for extend relationships

Extend relationship is depicted by a **dashed arrow** with a **«extend» stereotype.**



Generalization relationship

Generalization relationship are used to represent the **inheritance between**

- **use cases** and
- **actors**

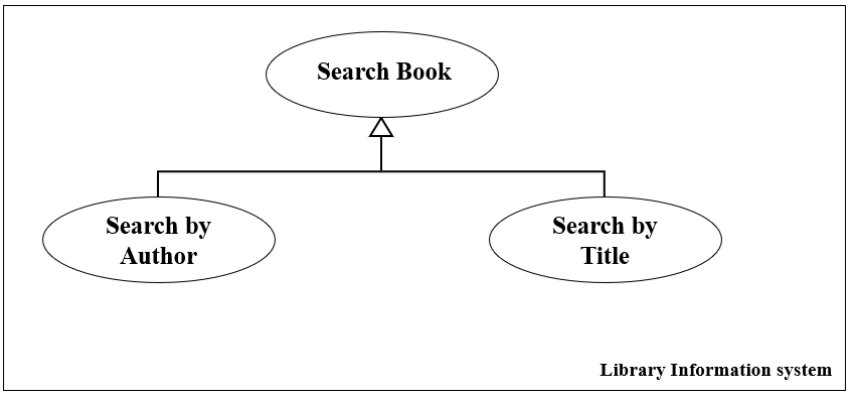
Generalization relationship between use cases

Here a **child use case inherits the behavior and meaning of the parent use case**. The child may **add or override** the behavior of the parent.

Notation for generalization relationships

Generalization relationship is depicted by a arrow **from the derived use case to the base use case.**

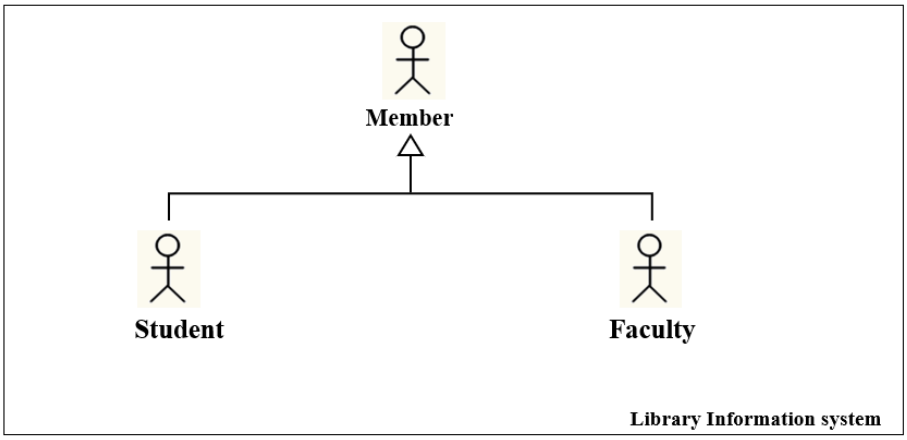
Example- Search Book use case with search by author name or book title



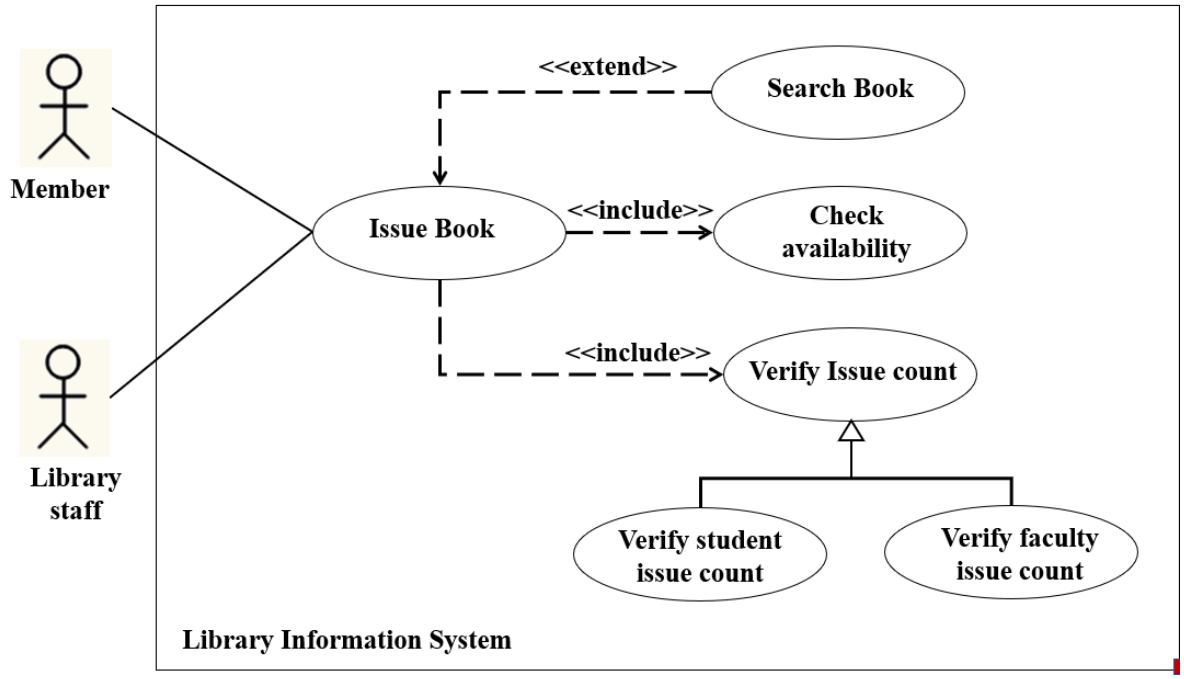
Generalization relationship between actors

The **generalization of actor** refers to the **relationship** which exist **between** two **actors** and which shows that one **actor** (descendant) inherits the role and properties of another **actor** (ancestor).

Example - a library member may be student and faculty



Use case diagram showing "Issue book" use case (scenario)



DATA FLOW DIAGRAM

A **data flow diagram** (**DFD**) is a **graphical representation of the "flow" of data** through an information system, modelling its process aspects.

A DFD is often used as a **preliminary step to create an overview of the system without going into great detail**, which can later be elaborated.

A DFD shows **what kind of information** will be input to and output from the system, **how the data will advance** through the system, and **where the data will be stored**.

It **does not show information about** process timing or whether processes will operate in sequence or in parallel.

DFD can be drawn to represent the system of different levels of abstraction.

Level-0 DFD (context diagram):

It is also known as a **context diagram**. It's designed to be an **abstraction view**, showing the **system as a single process** with its relationship to **external entities**. It represents the entire system as a single circle with input and output data indicated by incoming/outgoing arrows.

Level-1 DFD:

In level-1 DFD, the context diagram is decomposed into **multiple sub-processes**. In this level, we highlight the main functions of the system and **breakdown** the high-level process of level-0 DFD **into sub-processes**.

Level-2 DFD:

Level-2 DFD goes one step **deeper into parts** of level-1 DFD. It can be used to plan or record the **specific/necessary detail about the system's functioning**.

Graphical notations for Data Flow Diagram

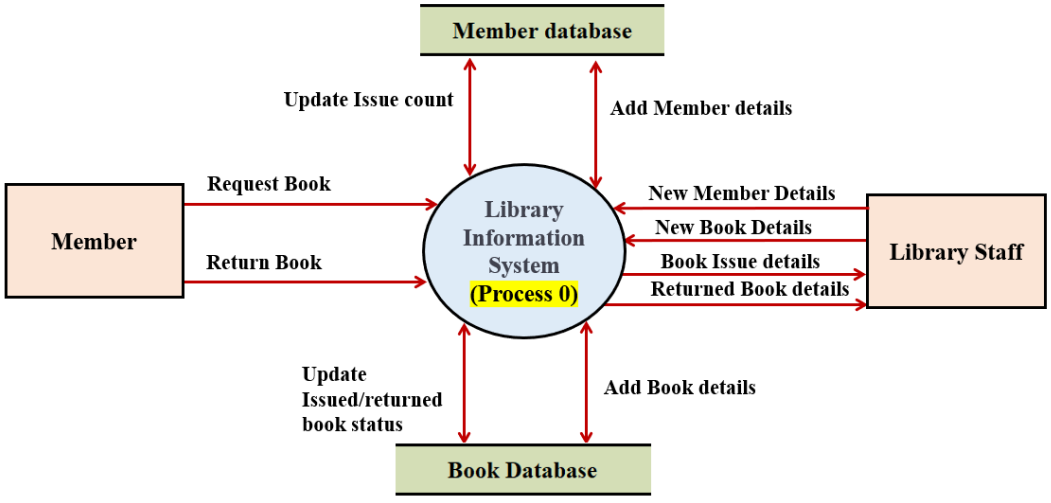
Term	Notation	Explanation
External entity	Passenger	External entities are represented by a **rectangle** and the name of the external entity is written inside the rectangle.
Process	Reservation	Processes are represented by **circle** and name of the process is written inside the circle.
Data store	Passenger Data	A left-right **open rectangle** is denoted as data store; name of the data store is written inside the shape
Data flow	Passenger details →	Data flow is represented by a **directed arrows** with its data name.

DFD for Library Information System

For level-0 DFD (Context Diagram) -

- External entities are
  - **Member**
  - **Library staff**
- Data stores are
  - **Member database**
  - **Book database**
- **Process 0** is **Library Information System**.

Level-0 DFD (context diagram)

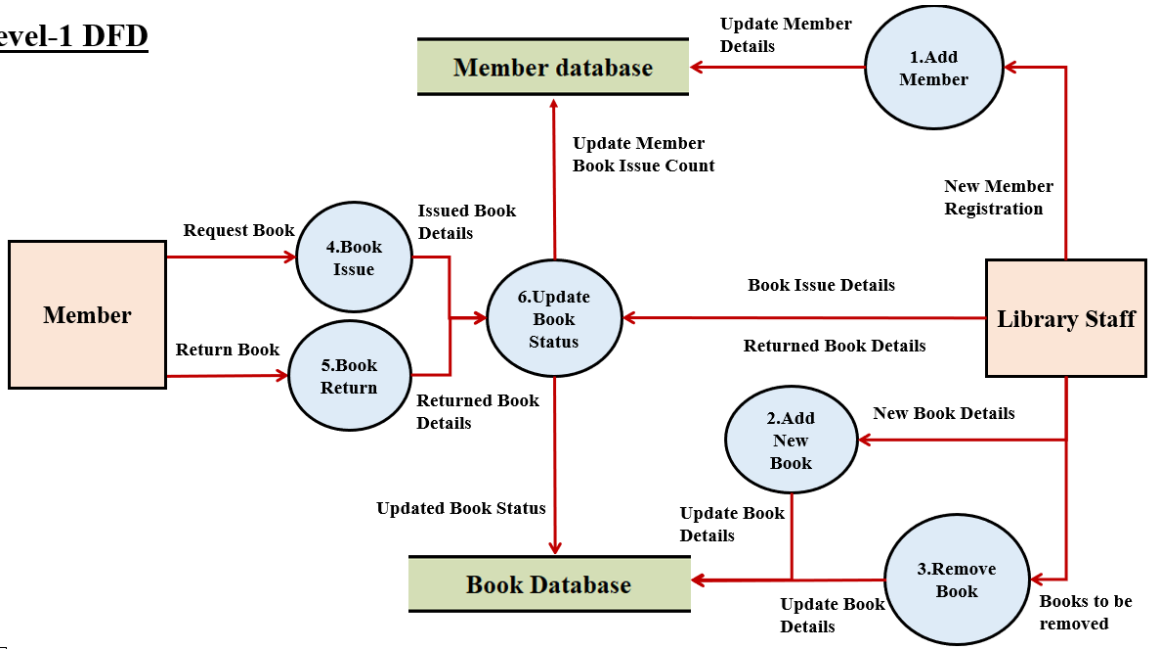


For level-1 DFD -

Sub-processes for "process 0" (Library Information System) may be

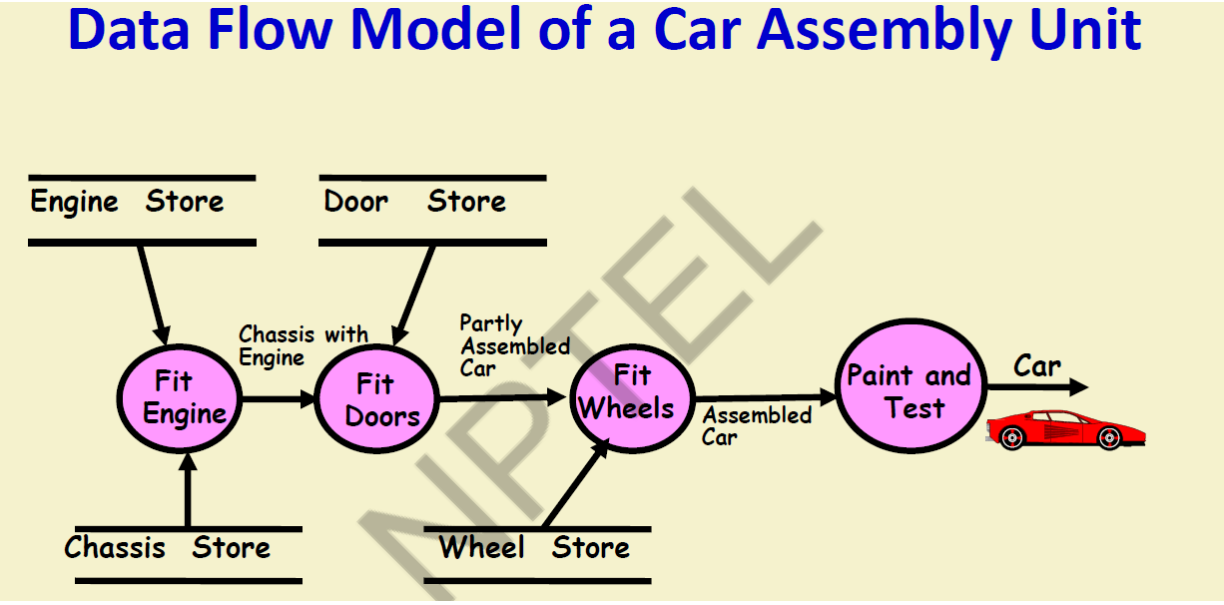
- **Add member**
- **Add book**
- **Remove book**
- **Issue book**
- **Return book**

Level-1 DFD



References:

- Roger Pressman, "Software Engineering: A Practitioner's Approach", McGraw-Hill Publications
- http://moodle.autolab.uni-pannon.hu/Mecha_tananyag/szoftverfejlesztesi_folyamatok_angol/ch07.html
- https://www.geeksforgeeks.org/functional-vs-non-functional-requirements/
- https://www.softwaretestinghelp.com/requirements-elicitation-techniques/
- https://www.umsl.edu/~sauterv/analysis/F2015/Requirement%20Gathering%20Methods.html.htm
- http://vlabs.iitkgp.ac.in/se/
- https://enfocussolutions.com/actors-and-users/
- http://vlabs.iitkgp.ac.in/se/8/case_study/
- https://www.geeksforgeeks.org/levels-in-data-flow-diagrams-dfd/



https://onlinecourses.nptel.ac.in/noc22_cs106/unit?unit=25&lesson=31